

Machine Problem: Basic Implementation of a Communication System in MATLAB

Ruz Brianne Godoy
Faculty of Engineering
University of Santo Tomas
Manila, Philippines
ruzbrianne.godoy.eng@ust.edu.ph

Sherman Lloyd Ko
Faculty of Engineering
University of Santo Tomas
Manila, Philippines
shermanlloyd.ko.eng@ust.edu.ph

Jared Asher Mananguit
Faculty of Engineering
University of Santo Tomas
Manila, Philippines
jaredasher.mananguit.eng@ust.edu.ph

Sherwin Miguel C. Mapaye
Faculty of Engineering
University of Santo Tomas
Manila, Philippines
sherwinmiguel.mapaye.eng@ust.edu.ph

Jose Anton Medenilla
Faculty of Engineering
University of Santo Tomas
Manila, Philippines
joseanton.medenilla.eng@ust.edu.ph

Abstract - This project presents the implementation of the Quadrature Amplitude Modulation (QAM) system for transmitting and reconstructing audio signals, a key technique in modern communication. The system was operated with a sampling frequency of 300kHz and a carrier frequency of 100kHz. The system was divided in the segments which includes, signal acquisition, amplitude normalization, modulation, coherent demodulation, and zero-phase filtering. Results show that the In-phase and Quadrature components were successfully separated with high fidelity, preventing the crosstalk often associated with phase distortion. Time-domain analysis confirmed that the recovered waveforms matched the original inputs, and subjective listening tests revealed no audible degradation after the signal was downsampled back to its original rate. Overall, the study highlights QAM's capability to deliver high-quality audio communication and its importance to current telecommunications applications.

Keywords - demodulation, filtering, modulation, quadrature amplitude modulation (QAM), resampling, sampling, signal processing, communication systems

I. INTRODUCTION

The efficient and reliable transmission of information is a fundamental challenge in today's modern communication systems. Digital communication, in particular, needs advanced techniques to maximize spectral efficiency while maintaining low error rates over noisy channels. [1, 2]. This project aims to introduce the mathematical

model and practical implementation of a complete digital communication system using MATLAB, with a focus on the principles of passband data transmission [3].

The technology at the core of this simulation is Quadrature Amplitude Modulation (QAM), a powerful modulation scheme that achieves high data rates by varying both the amplitude and phase of the carrier signal.[4]. The QAM transmitter is structured with two separate branches—the in-phase (I) and quadrature (Q) branches—that use carriers 90° out of phase [5]. This setup allows two independent bitstreams to be transmitted simultaneously over the same bandwidth, significantly improving transmission efficiency [6].

The performance of any communication system is critically limited by the impairments introduced by the channel. In this project, the system's performance is evaluated under the presence of the Additive White Gaussian Noise (AWGN), a widely accepted model for thermal noise in electronic devices [4, 8]. Previous studies have comprehensively analyzed QAM performance over the AWGN channel, noting the trade-off between power efficiency and spectral efficiency [5, 6, 7]. The ability to accurately model the Bit Error Rate (BER) of the QAM signal as a function of the signal-to-noise ratio (E_b/N_0) is central to validating the system design [7, 8].

The implementation tasks cover the entire communication chain, starting with signal acquisition and sampling rate conversion to prepare the audio inputs for digital processing [9]. Subsequent stages include the mathematical implementation of QAM modulation, transmission across the AWGN channel, and coherent demodulation to recover the original signals, providing a comprehensive analysis of the communication process [3]. The findings contribute to a deeper understanding of how practical implementation constraints, such as synchronization and filtering, affect the theoretical performance limits of QAM systems.

II. IMPLEMENTATION

The design methodology involves implementing a QAM system through a structured process in MATLAB. This approach was guided by established communication theories and principles, particularly those outlined in the works of Haykin (2012) and Carlson et al. (2010). The process is organized into five stages:

A. Signal Acquisition and Processing

The project began by preparing two distinct audio signals representing the in-phase and quadrature data streams to be transmitted. Before the signals were processed, the system parameters were initialized based on user input:

Parameter	Symbol	Value
Carrier Frequency	f_c	100kHz
Target Sampling Frequency	$F_{S,new}$	$3 \times f_c$
Filter Cutoff	F_{cut}	$F_{S,orig}/5$

The acquisition process, encapsulated in the `load_input` function, involved three key steps along with its corresponding line of MATLAB code.:

1.) Loading and Mono-conversion

The audio files were loaded using `audioread`. To ensure consistency for the single-channel processing, any stereo inputs were converted to mono by averaging the channels.

```
[x1, Fs1] = audioread(student1);
[x2, Fs2] = audioread(student2);
```

```
if size(x1,2) > 1
    x1 = mean(x1, 2);
end

if size(x2,2) > 1
    x2 = mean(x2, 2);
end
```

2.) Rate Conversion

The sampling frequencies of the two inputs F_{s1} , F_{s2} were compared. To preserve signal quality, both signals were resampled to the higher of the two rates.

```
if Fs1 ~= Fs2
    Fs_orig = max(Fs1, Fs2);
    x1 = resample(x1, Fs_orig, Fs1);
    x2 = resample(x2, Fs_orig, Fs2);
else
    Fs_orig = Fs1;
end
```

3.) Synchronization

To ensure synchronous transmission duration, a nested `padzero` function was utilized to append trailing zeros to the shorter signal, ensuring both x_1 and x_2 has an equal length L , as misaligned signals could cause some errors during the modulation process [1].

```
[x1pad, x2pad] = padzero(x1, x2);

length_test = size(x1pad, 1) == size(x2pad, 1);

if length_test == 1
    L = size(x1pad, 1);
else
    error('The lengths of the padded signals
do not match.');
```

```
end

function [x1pad, x2pad] = padzero(x1, x2)

    len_x1 = size(x1, 1);
    len_x2 = size(x2, 1);
    max_len = max(len_x1, len_x2);

    x1pad = [x1; zeros(max_len - len_x1,
size(x1, 2))];
    x2pad = [x2; zeros(max_len - len_x2,
size(x2, 2))];
end
```

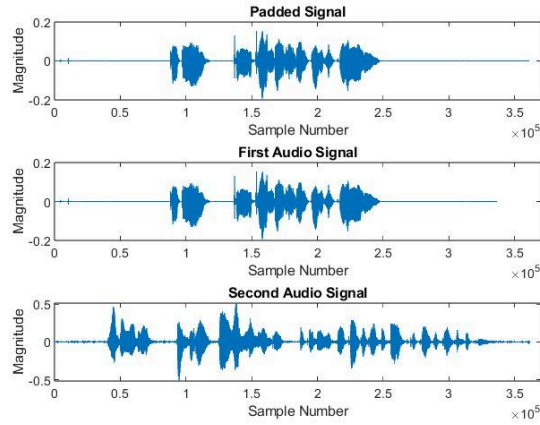


Figure 1. Plot of Comparison of Padded Signal with Original Signals

B. Signal Resampling (Upsampling)

To facilitate modulation at the high carrier frequency f_c , the baseband signals required upsampling to the target frequency $F_{s,new}$. This was executed in the *resample_signal* function:

```
>>function[x1_resampled,x2_resampled]=
resample_signal(x1_pad,      x2_pad,      Fs_new,
Fs_orig)
```

1.) Rational Approximation:

The MATLAB *rat* function was employed to determine the optimal integer numerator (P) and denominator (Q) such that the resampling ratio approximated the target:

$$\frac{P}{Q} \approx \frac{F_{s,new}}{F_{s,orig}}$$

```
>> [P, Q] = rat(Fs_new / Fs_orig);
```

2.) Polyphase Resampling:

The *resample* function applied these factors to the padded signals, generating high-resolution versions $x_{1,resampled}$ and $x_{2,resampled}$ suitable for mixing with the carrier.

```
>> x1_resampled = resample(x1_pad, P, Q);
>> x2_resampled = resample(x2_pad, P, Q);
```

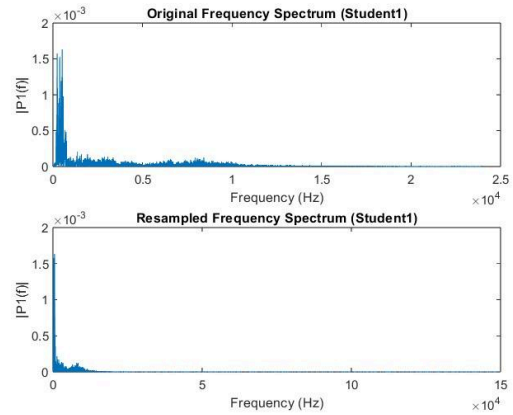


Figure 2.1. Plot of Comparison of Original and Resampled Audio (Student1)

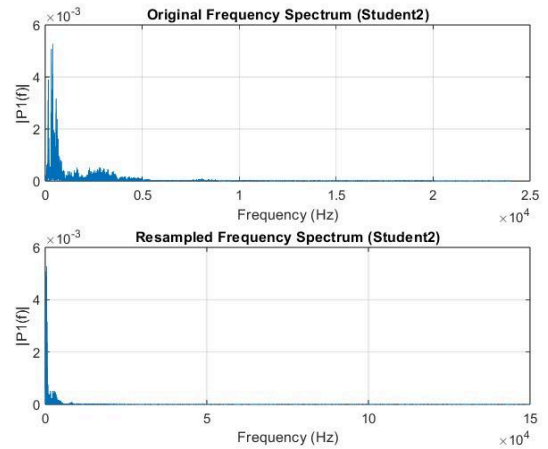


Figure 2.2. Plot of Comparison of Original and Resampled Audio (Student2)

C. QAM Modulation Stage

The modulation implemented in *qam_modulation* combines the two audio files into a single passband signal.

```
function rx = qam_modulation(x1_resampled,
x2_resampled, fc, t)
```

1.) Carrier Generation:

The In-phase (C_s) and In-quadrature (C_Q) carriers were generated as orthogonal sinusoids using the time vector t .

```
>> c_S = cos(2*pi*fc.*t);  
>> c_Q = sin(2*pi*fc.*t);
```

2.) Amplitude Normalization:

In order to prevent numerical instability and clipping during the addition of signals, the two inputs were normalized by the maximum absolute amplitude found in the signal set.

```
>> mabs=max([max(abs(x1_resampled)),  
max(abs(x2_resampled)), 1e-12]);  
>> x1n = x1_resampled./mabs;  
>> x2n = x2_resampled./mabs;  
>> rx = x1n .* c_S + x2n .* c_Q;
```

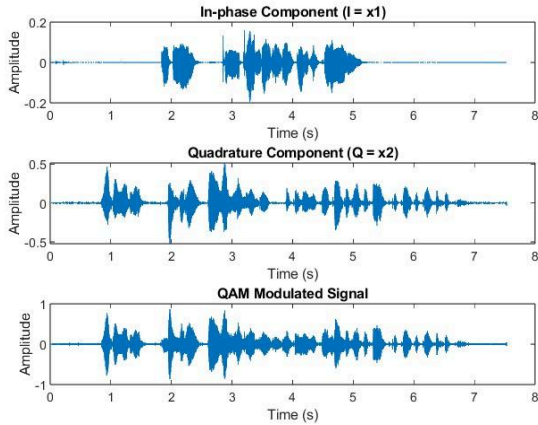


Figure 3. Plot of Quadrature Modulated Signal, In Phase Component, In Quadrature Component

3.) Signal Composition:

The final QAM signal $x_{AM}(t)$ was constructed by summing up the modulated components.

```
>> x_AM = x1n .* c_S + x2n .* c_Q;
```

D. QAM Demodulation Stage

The receiver stage, *qam_demodulation*, which performed coherent detection to retrieve the passband information from the received signal $rx(t)$.

1.) Product Detection:

The system generated local oscillators, lo_i , lo_q , synchronized with the transmitter carrier's frequency f_c . The received signal was then multiplied to these local oscillators to separate the in-phase and in-quadrature components.

2.) Zero-phase Filtering:

A Butterworth Low-Pass Filter was designed utilizing the *butter* function. To ensure the high fidelity of the signal, the *filtfilt* function was used instead of the standard filter, thereby eliminating phase distortion during signal recovery.

```
>> function [y1, y2, yfilt1, yfilt2] =  
qam_demodulation(rx, fc, fcut, Fs_new, t, f_ord)  
>> loi = cos(2*pi*fc*t);  
>> loq = sin(2*pi*fc*t);  
>> y1 = rx .* loi;  
>> y2 = rx .* loq;  
>> wn = 2*fcut/Fs_new;  
>> [b, a] = butter(f_ord, wn);  
>> yfilt1 = filtfilt(b, a, y1);  
>> yfilt2 = filtfilt(b, a, y2);
```

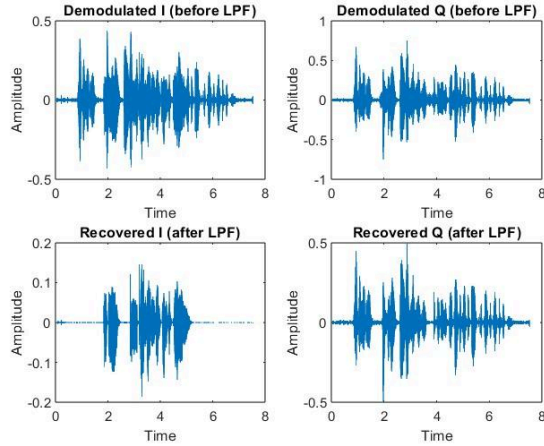


Figure 4. Plot of Comparison Between Demodulated Signals and Recovered Signals using LPF

E. Resampling the Signal

The final stage reversed the initial signal processing to prepare the audio for playback. This involved downsampling the signal and normalizing the amplitude before saving the files.

1.) Downsampling Ratio Calculation

The `downsample_signal` function calculated the inverse rational factors (P, Q) using the `rat` function and applied `resample` to restore the signals to their original sampling rate ($F_{s,orig}$)

$$\frac{P}{Q} \approx \frac{F_{s,orig}}{F_{s,new}}$$

```
>> function [y1_down, y2_down] =  
downsample_signal(y1, y2, Fs_new, Fs_orig)
```

```
>> [P, Q] = rat(Fs_orig / Fs_new);
```

2.) Polyphase Resampling:

The `resample` function utilized these factors to the padded signals, producing high-resolution versions $y_{1,down}$ and $y_{2,down}$ suitable for mixing with the carrier.

```
>> y1_down = resample(y1, P, Q);
```

```
>> y2_down = resample(y2, P, Q);
```

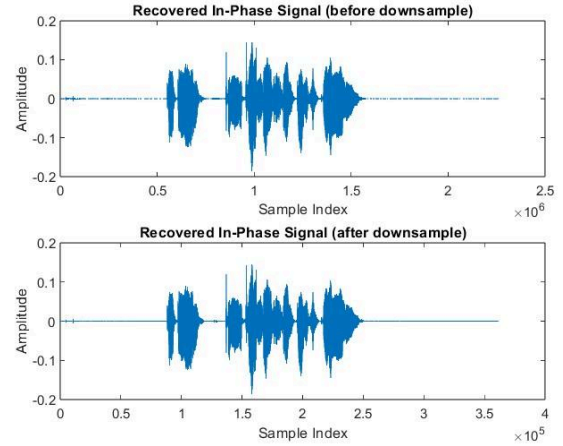


Figure 5.1 Plot of Downsampled In-Phase Signal

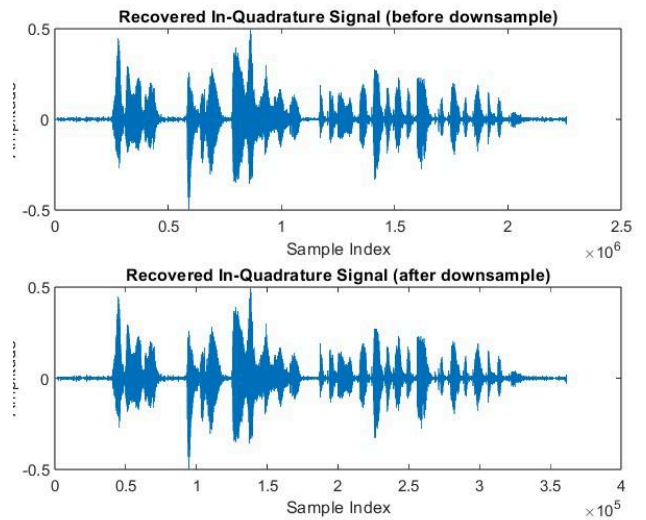


Figure 5.2 Plot of Downsampled In-Quadrature Signal

F. Normalizing Audio Signals and Exporting Recovered Audio

Before the final files were generated, the recovered signals were normalized to ensure they remained within the standard digital audio range of $[-1, 1]$. By dividing each signal by its maximum absolute amplitude, the system maximized the dynamic range while preventing digital clipping

```
audio_out1 = y1_down;  
audio_out1 = audio_out1 / max(abs(audio_out1));  
audio_out2 = y2_down;  
audio_out2 = audio_out2 / max(abs(audio_out2));
```

```

output_filename1 = '3ECEB_Group
No.6_Godoy,Ruz_OUTPUT.wav';
output_filename2 = '3ECEB_Group
No.6_Medenilla,Anton_OUTPUT.wav';

```

Finally the processed signals were exported as .wav files using the *audiowrite* function. This step reconstructed the audio at the original sampling frequency $F_{s, \text{orig}}$, allowing external playback.

```

audiowrite(output_filename1, audio_out1,
Fs_orig);
audiowrite(output_filename2, audio_out2,
Fs_orig);

```

III. ANALYSIS AND DISCUSSION OF RESULTS

In testing the implementation of the MATLAB code, the carrier frequency was set as 100kHz. With a standard audio bandwidth up to 20kHz, a 100kHz carrier will be able to ensure clear signal separation without demanding excessive computational power. Furthermore, using this carrier frequency will lessen the computation and processing required from MATLAB while still remaining sufficient to properly recover the original data signal.

As both audio files have the same sample rate of 48kHz, no resampling was made. Furthermore, the first audio signal was found to be stereo (two channels), it was converted into a single channel. The matching of sampling index of Audio1 and Audio2 by padding zeros was found to be successful as shown in Figure 1. As the first audio signal was found to be shorter, it was the one padded with the extra zeros. Audio 1 went from 337,008 samples to 361,464 which matched the longer audio signal of Audio 2.

The new sampling frequency was decided to be thrice the carrier frequency. This puts it way above the Nyquist Rate of $F_s = 2 \cdot F_{\text{max}}$ which is the minimum sampling frequency to avoid aliasing. By substantially exceeding this baseline, the design ensures that the upper sidebands are captured without distortion and creates a wider separation between spectral components, which greatly simplifies the task of filtering. Furthermore, when the Nyquist Rate was used, there was audible distortion within the recovered audio files.

Figure 3 provides a time-domain analysis of the QAM modulation experiment, illustrating how two separate audio channels are synthesized into a single transmission waveform. The input signals, displayed in the top two panels, show distinct amplitude variations; the In-phase signal (I) is relatively quiet with peaks around ± 0.2 , while the Quadrature signal (Q) is stronger, reaching amplitudes of ± 0.5 . The bottom panel reveals the modulated output, which represents the vector sum of these inputs carried by sine and cosine waves. This composite signal exhibits a much denser waveform structure due to the high carrier frequency and achieves a peak magnitude of roughly ± 1.0 . This shows and confirms the modulation process was successful as two signals were successfully encoded in a single bandwidth.

The QAM demodulation stage successfully retrieved the information signal through coherent detection, where the received signal was multiplied by using the local oscillators to separate the In-phase and Quadrature components. To recover the original signal fidelity, a Butterworth Low-Pass Filter was applied using a zero-phase technique, which recovered the message signals and removed any unwanted noise. The result of this filtering is demonstrated in the lower subplots of Figure 4, where the clean, smoothed waveforms confirm that the audio signals were accurately reconstructed and separated without visible phase distortion.

A 4th-order Butterworth Low-Pass Filter was applied using a zero-phase method. It was chosen as it provides a good balance to remove the high frequency signals while avoiding any distortion.

The final phase of the signal processing reverting the demodulated signals to their original sampling rates to have proper audio playback. As shown in Figure 5.1 and Figure 5.2, there is a significant decrease in the number of samples from around 2.2 million samples to only around 360,000 samples. Despite this reduction in sample count, the waveform in both figures is still identical to the waveform before downsampling meaning that the resampling process successfully preserved the signal fidelity while eliminating the redundant simulation data.

After exporting the normalized recovered waveform to an *audiofile*, the two audio signals were properly demodulated and separated. There was no audible difference between the original audio and the recovered audio signal.

IV. CONCLUSION AND RECOMMENDATION

The project successfully demonstrated the implementation of a complete QAM communication system in MATLAB for transmitting and recovering two independent audio signals over a shared bandwidth. The system was designed following established communication principles from Haykin [1] and Carlson et al.[4], incorporating signal acquisition and processing, upsampling, QAM modulation, coherent demodulation with low-pass filtering, and downsampling for audio reconstruction.

The results validated the effectiveness of the QAM technique in achieving a high spectral efficiency by encoding two separate audio—the In-phase (I) and Quadrature (Q) components—onto orthogonal carriers. Using a 100 kHz carrier frequency and a target sampling rate set at three times the carrier frequency, the system successfully exceeded the Nyquist rate requirement, thereby preventing aliasing and ensuring accurate signal reconstruction. The choice of a 4th-order Butterworth low-pass filter with zero-phase implementation proved effective in removing high-frequency components introduced during the modulation process while preserving the fidelity of the recovered audio signals.

The modulation stage successfully combined the normalized audio signals into a composite QAM waveform, as evidenced by the time-domain plots showing distinct amplitude characteristics of the I and Q components. The demodulation process, employing coherent detection with synchronized local oscillators, accurately separated the two audio channels without observable phase distortion. Following downsampling from approximately 2.2 million samples to 360,000 samples, the recovered audio files exhibited no audible difference from the original recordings, confirming that the implementation maintained signal integrity throughout the entire communication chain.

While the current implementation demonstrates QAM transmission under ideal conditions, future work could incorporate Additive White Gaussian Noise (AWGN) channel modeling to evaluate the system's performance under realistic noise conditions. This would enable comprehensive Bit Error Rate (BER) analysis as a function of signal-to-noise ratio (E_b/N_0), providing quantitative metrics for assessing system robustness and reliability [10]. By simulating transmission over AWGN channels at varying noise levels, developers can establish performance curves that characterize the trade-offs between spectral efficiency and noise immunity, ultimately validating the system design against real-world deployment scenarios [11].

The fixed 4th-order Butterworth filter performed adequately in this implementation; however, an adaptive filtering approach that adjusts the filter order based on the carrier frequency and signal characteristics could optimize performance across varying operational conditions. Such adaptive mechanisms would dynamically modify filter parameters—including order, cutoff frequency, and coefficient values—in response to real-time signal conditions, enabling the receiver to maintain optimal noise suppression while minimizing signal distortion [12]. Implementing algorithms such as the Least Mean Square (LMS) or Recursive Least Squares (RLS) would allow the system to continuously estimate and compensate for channel impairments, resulting in improved recovery of the demodulated audio signals and enhanced overall system performance across different audio source characteristics and environmental noise profiles.

Additionally, transitioning the MATLAB simulation to hardware platforms such as Field-Programmable Gate Array (FPGA) or Digital Signal Processing (DSP) processors would validate the practical applicability of the designed system and expose additional challenges related to timing synchronization, quantization effects, and computational constraints. FPGA implementations offer particular advantages through reconfigurable architectures that can simultaneously support multiple modulation schemes, while real-time DSP deployment would provide insights into the computational overhead of maintaining carrier phase coherence and symbol-rate synchronization during continuous operation [13].

Overall, this project has effectively demonstrated that QAM is a robust and bandwidth-efficient modulation scheme suitable for high-quality audio communication. The MATLAB simulation provided practical insights into the trade-offs between sampling frequency selection, filter design, and signal quality—knowledge that is directly applicable to real-world telecommunications systems. Applying the recommendations outlined in this report—such as incorporating AWGN channel modeling, implementing adaptive filtering techniques, and FPGA or DSP implementation—could significantly advance the system's capability to operate reliably in noisy environments and demanding real-world applications, ultimately bridging the gap between theoretical communication principles and practical deployment scenarios.

V. REFERENCES

- [1] S. Haykin and M. Moher, *An Introduction to Analog and Digital Communications*. Hoboken, NJ, USA: Wiley, 2012..

[2] S. Haykin, *Communication Systems, 4th ed.* Hoboken, NJ, USA: Wiley, 2001..

[3] J. Santos, *ECE21118-Communication System and Theory: Machine Problem.* Project Guide/Module, 2025.

[4] A. B. Carlson et al., *Communication Systems: An Introduction to Signals and Noise in Electrical Communication*, 5th ed. New York, NY, USA: McGraw Hill, 2010..

[5] R. V. K. A. V. S. N. V. Reddy and R. J. R. A. V. S. V. S. V. Reddy, "Performance Analysis of Rectangular 16-QAM in AWGN Channel," *International Journal of Engineering Research & Technology (IJERT)*, vol. 8, no. 12, pp. 110-113, Dec. 2019.

[6] T. I. Jirnadu et al., "Comparative Analysis of QAM Modulation Techniques over AWGN and Fading Channels," *Advances in Science and Technology*, vol. 107, pp. 194-200, Jun. 2021. doi: 10.4028/www.scientific.net/AST.107.194.

[7] R. D. B. C. et al., "A MATLAB Simulink Study on the Performance of QAM Modulation Scheme in AWGN, Rayleigh, and Rician Fading Channels: BER Analysis," *Irish Interdisciplinary Journal of Science & Research (IJSR)*, vol. 7, no. 2, pp. 34-41, Apr.-Jun. 2023.

[8] A. R. V. K. Reddy et al., "Performance Evaluation of Digital Modulation Techniques In Awgn Communication Channel," *International Journal of Engineering Research & Technology (IJERT)*, vol. 2, no. 5, pp. 3173-3178, May 2013.

[9] J. F. C. R. et al., "Sampling Rate Conversion Techniques- A Review," *2021 4th International Conference on Recent Trends in Computer Science and Technology (ICRTCST)*, pp. 267-272, 2022. doi: 10.1109/ICRTCST53715.2022.9781914.

[10] P. Kogias, K. Ovaliadis, and F. Kogia, "COMPARATIVE PERFORMANCE EVALUATION OF M-ARY QAM MODULATION SCHEMES USING SIMULINK AND BERTool," *ARPJ Journal of Engineering and Applied Sciences*, vol. 12, no. 22, 2017, Available: https://www.arpnjournals.org/jeas/research_papers/rp_2017/jeas_1117_6487.pdf

[11] MathWorks, "Channel Mode I," *Mathworks.com*, 2025. <https://www.mathworks.com/discovery/channel-model.html>

[12] S. Chaudhary, "Matlab Based Design of Adaptive Filters Using Least Pth Norm: FIR vs IIR," *International Journal of Signal Processing Systems*, vol. 2, 2014, doi: <https://doi.org/10.12720/ijsp.2.1.60-63>.

[13] A. Gomez and A. Menon, "FPGA IMPLEMENTATION OF QAM," 2016. Available: https://www.digitalxplore.org/up_proc/pdf/226-146285922868-70.pdf

VI. DISTRIBUTION OF TASKS

Authors	Tasks
Sherman Lloyd Ko	Coding (QAM Modulation) Implementation
Ruz Brianne Godoy	Audio Conclusion and Recommendation References
Jared Asher Mananguit	Coding (Resampling, QAM Demodulation, Downsampling) Implementation Analysis and Discussion of Results
Sherwin Miguel Mapaye	Coding (Loading Signals) Abstract Implementation
Jose Anton Medenilla	Audio Introduction Implementation References

VII. APPENDIX

MATLAB Codes:

Task 1: Loading the signal

```
function [x1pad, x2pad, Fs_orig, x1,x2,Fs1,Fs2]
= load_input(student1, student2)
[x1, Fs1] = audioread(student1);
[x2, Fs2] = audioread(student2);
disp(['Sampling Frequency of Student 1
Recording: ', num2str(Fs1)]);
disp(['Sampling Frequency of Student 2
Recording: ', num2str(Fs2)]);

% If input signal is stereo (dual channel),
convert signal to mono (single channel)
% Checks the stereo audio and get the
average for
% both left and right channels and
converts to single channel
```



```

if size(x1,2) > 1
    x1 = mean(x1, 2);
end

if size(x2,2) > 1
    x2 = mean(x2, 2);
end
% if there is difference in Fs, this will
handle the mismatch
if Fsl ~= Fs2
    % Choose a common sampling frequency
    (higher of the two)
    Fs_orig = max(Fs1, Fs2);
    % Resample both signals to the common
    sampling frequency
    x1 = resample(x1, Fs_orig, Fsl);
    x2 = resample(x2, Fs_orig, Fs2);
else
    Fs_orig = Fsl; % Original sampling
    frequency
end
% Using the function 'padzero'
[x1pad, x2pad] = padzero(x1, x2);

% Get the length of the padded signals
length_test = size(x1pad, 1) == size(x2pad,
1);
if length_test == 1
    L = size(x1pad, 1);
else
    error('The lengths of the padded signals do
not match.');
```

end

% Nested function to pad zeros to the
signals

```

function
    [x1pad, x2pad] = padzero(x1, x2)
    % Determine the row size of x1, x2
    len_x1 = size(x1, 1);
    len_x2 = size(x2, 1);
    % Determine the largest value between x1
    & x2
    max_len = max(len_x1, len_x2);

    % Pad zeros to match the length of the
    longer signal
    x1pad = [x1; zeros(max_len - len_x1,
size(x1, 2))];
    x2pad = [x2; zeros(max_len - len_x2,
size(x2, 2))];
end
End
```

Task 2: Resampling Audio Signals

```

%% This function upsamples the two input signals
into a higher sampling frequency
function [x1_resampled, x2_resampled] =
resample_signal(x1_pad, x2_pad, Fs_new, Fs_orig)

%% Function Inputs
% Fs_new = Target Sampling Frequency
% Fs_old = The original sampling frequencies
% x1 & x2 = Audio input signals
%% Function Outputs
% x1_resampled, x2_resampled = resampled signal
to the target sampling freq
    % Computing for resampling ratio
```

```

[P, Q] = rat(Fs_new / Fs_orig);
% Resampling both signals
x1_resampled = resample(x1_pad, P, Q);
x2_resampled = resample(x2_pad, P, Q);
end
```

Task 3: QAM Modulation

```

function rx = qam_modulation(x1_resampled,
x2_resampled, fc, t)
% Simple QAM modulation (no pulse shaping).
x1,x2 and t must be same length.
% x_AM = x1.*cos(2*pi*fc*t) -
x2.*sin(2*pi*fc*t); % sign convention variant
% We'll use plus for Q term added with sin:
c_S = cos(2*pi*fc*t);
c_Q = sin(2*pi*fc*t);
% Normalize amplitude to avoid clipping
mabs= max([max(abs(x1_resampled)),
max(abs(x2_resampled)), 1e-12]);
x1n = x1_resampled./mabs;
x2n = x2_resampled./mabs;
% QAM composite
rx = x1n .* c_S + x2n .* c_Q;
end
```

Task 4: QAM Demodulation

```

%% This function performs demodulation using
low-pass filter
function [y1, y2, yfilt1, yfilt2] =
qam_demodulation(rx, fc, fcut, Fs_new, t,f_ord)
%% Function Inputs:
%rx = received QAM signal
%fc = carrier frequency used in modulation
%fcut = cutoff frequency
%Fs = sampling frequency
%t = time vector
%f_ord = order of filter
%% Function Outputs:
% y_1 & y_2 = demodulated I and Q before
filtering
%y_filt_1 & y_filt_2 = demodulated I and Q
after low-pass filtering
    % Simulate Local Oscillators
    loi = cos(2*pi*fc*t); % In-Phase
    loq = sin(2*pi*fc*t); % In-Quadrature
    % Simulate Product Detection
    % Multiply the received signal to the local
    oscillators
    y1 = rx .* loi; % In-phase branch
    y2 = rx .* loq; % Quadrature branch
    % Creating a low pass filter
    wn = 2*fcut/Fs_new;
    [b, a] = butter(f_ord, wn);

    % Apply the filter
    yfilt1 = filtfilt(b, a, y1);
    yfilt2 = filtfilt(b, a, y2);
end
```

```

%% This function downsamples the two input
signals into the original sampling frequency
```

Task 5: Downsampling Recovered Signals

```

function [y1_down, y2_down] =
downsample_signal(y1, y2, Fs_new, Fs_orig)
%% Function Inputs:
%   y1, y2 = filtered audio signals
%   Fs_new = Sampling Freq after Resampling
%   Fs_orig = Original Sampling Frequency of
the audio signals
%% Function Outputs:
%   y1_down, y2_down - signals resampled to
original Fs
    % Computing resample ratio
    [P, Q] = rat(Fs_orig / Fs_new);
    % Resampling the signals
    y1_down = resample(y1, P, Q);
    y2_down = resample(y2, P, Q);
end

```

```

audiowrite(output_filename1, audio_out1,
Fs_orig);
audiowrite(output_filename2, audio_out2,
Fs_orig);

```

Script File Implementation for All Functions

```

%Audiofile input
student1 = '3ECEB_Group
No.6_Godoy,Ruz_INPUT.mp3';
student2 = '3ECEB_Group
No.6_Medenilla,Anton_INPUT.m4a';

%Input Carrier Frequency and Order of Filtering
fc = input('Input Carrier Freq (Hz): ');
%Set Target Frequency
Fs_new = 3*fc;

%Call function to load and process inputs
[x1_pad, x2_pad, Fs_orig,x1,x2,Fs1,F2] =
load_input(student1, student2);

%Resample the Signals to Target Frequency
[x1_resampled, x2_resampled] =
resample_signal(x1_pad, x2_pad, Fs_new,
Fs_orig);

%Creating Time Vector
L = length(x1_resampled);
t = (0:L-1) / Fs_new;
t = t(:);

%Modulate the Two signals
rx = qam_modulation(x1_resampled, x2_resampled,
fc, t);
%Set cutoff freq and Filter Order
fcut = Fs_orig/5;
f_ord = 4;

%Demodulating the QAM Modulated Signal
[y1, y2, yfilt1, yfilt2] = qam_demodulation(rx,
fc, fcut, Fs_new, t,f_ord);

%Downsampling demodulated signal onto the
original sampling frequency
[y1_down, y2_down] = downsample_signal(yfilt1,
yfilt2, Fs_new, Fs_orig);

%Saving As Audio File
audio_out1 = y1_down;
audio_out1 = audio_out1 / max(abs(audio_out1));
audio_out2 = y2_down;
audio_out2 = audio_out2 / max(abs(audio_out2));

output_filename1 = '3ECEB_Group
No.6_Godoy,Ruz_OUTPUT.wav';
output_filename2 = '3ECEB_Group
No.6_Medenilla,Anton_OUTPUT.wav';

```